

AI Principles and Practice · Lecture 2

Supervised Learning

Week 2 · Phase I — Foundations and Ethics · LOs: LO2, LO3

Lecturer: Jonathan

Today's roadmap

1. **The supervised setup** — features, targets, and an honest split
2. **Regression** — straight lines, bent lines, and the price of flexibility
3. **Classification** — logistic, k-NN, trees, forests
4. **Loss and the bias–variance trade-off**
5. **Evaluation that doesn't flatter**
6. **Framing real problems** — the metric follows the question

One thread runs through all of it

A model is easy to build and easy to fool yourself with. Most of today is about the second half of that sentence — telling an honest result from a flattering one.

What you'll be able to do

Where this sits (LO2, LO3)

- **LO2** — weigh up competing ML methods and pick the one a problem actually needs
- **LO3** — build a model and, harder, read its evaluation with a sceptic's eye

By the end you should be able to

- Diagnose over- and underfitting from a learning curve, then say what to change
- Judge which metric a problem demands, and defend the choice
- Compare models on the terms that matter, not on accuracy alone
- Design an evaluation you'd trust with a real decision

What you're carrying in from Week 1

- The leakage-safe pipeline you built last week. Models drop straight into its final step.
- The same customers table — only now we predict **from** it rather than just describe it.
- The instinct to split first, and to trust the picture over any single number.

The shift

Week 1 asked “what is this data?”. Week 2 asks “what can it decide?” — and how would we know if the answer were wrong.

Section 1

The supervised setup

$$\hat{y} = f(x; \theta)$$

features x in, a prediction $\hat{y}$ out; learning means tuning θ so the two line up on data we have not yet seen

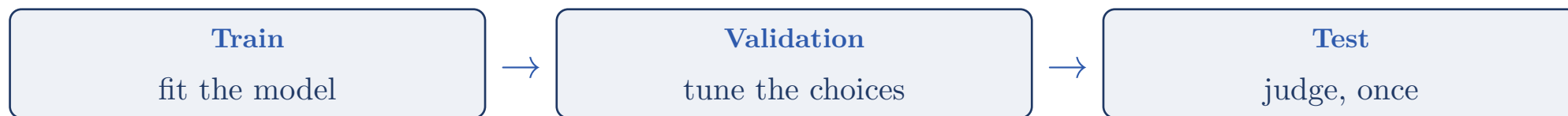
The parts

- **Features** x — what we know about a case
- **Target** y — what we want to predict
- **Parameters** θ — what the fitting adjusts

The point that matters

- Fitting the training data is trivial.
- The goal is **generalisation** — being right on the next customer, not this one.
- Memorising is not learning; it just looks like it until the test set.

Three splits, not two — and why



- **Train** fits the parameters.
- **Validation** is where you choose everything else: which model, which degree, which k , which λ .
- **Test** is spent once, at the very end, to estimate how you'll do in the wild.

The one rule

Every time you peek at the test set to make a decision, it stops being a test. Cross-validation (later) buys you honest tuning without spending it.

Regression or classification?

The quick answer: look at the target

- A **number** on a scale → regression (monthly_charge, demand, price).
- A **category** → classification (churn yes/no, fraud/not, which disease).

The better answer: look at the question

- The target is a **decision you make**, not a fact of the data.
- “How much will they spend?” and “Will they spend at all?” are different problems on the same table.

Choosing the target is already modelling. Get it wrong and the cleverest algorithm answers a question nobody asked.

This week's data: the same customers

targets.py

python

```
# Week 1's leakage-safe ColumnTransformer, reused
verbatim
prep = ColumnTransformer([
    ("num", num_pipe, ["age", "income", "tenure"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
     ["plan", "region", "satisfaction"]),
])
y_reg = df["monthly_charge"]    # a number ->
regression
y_clf = df["churn"]             # yes / no ->
classification
```

the two jobs

```
regression : predict monthly_charge
classification : predict churn
               (base rate = 15% leave)
```

One table, two questions. We'll carry both all the way to the evaluation.

Section 2

Regression

$$\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

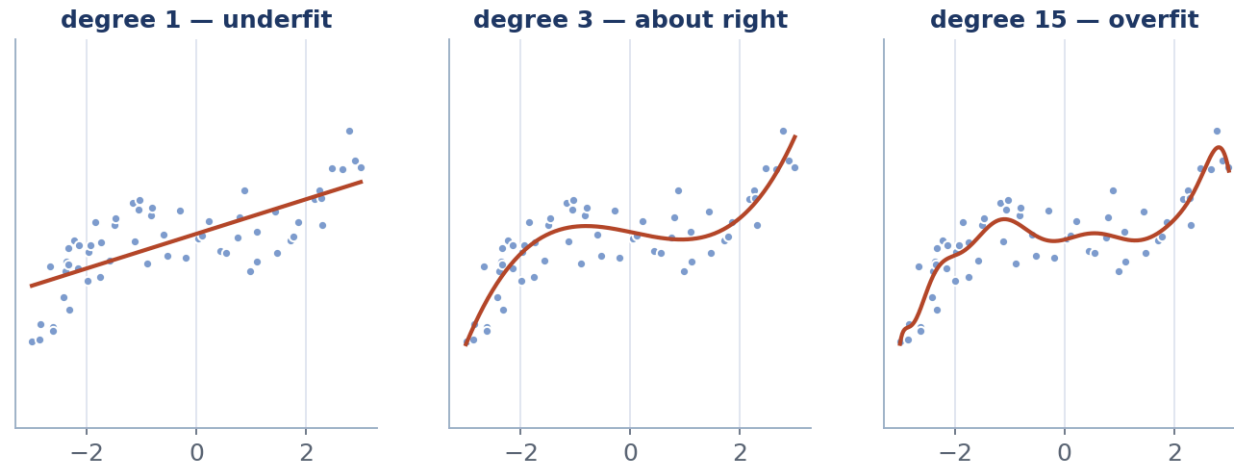
choose the β s to make the total squared error $\sum_i (y_i - \hat{y}_i)^2$ as small as possible

- Each β_j is the effect of one feature, holding the rest fixed.
- Least squares has a closed form — no iteration, and it's fast.
- It's the honest baseline: beat this before reaching for anything fancier.

Read it, don't just run it

A coefficient's **sign** and **size** are a claim about the world. If income has a negative charge coefficient, either the world is strange or your data is.

When a straight line won't do



The same points, three capacities. Degree 1 can't bend to the shape; degree 15 chases the noise; degree 3 catches the signal and ignores the rest.

Polynomial features reuse the **same** linear machinery on $x, x^2, x^3, \dots$. Flexibility is a dial, and turning it up is not free.

Regularisation: ridge and lasso

$$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j \beta_j^2 \quad \text{vs} \quad \min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j |\beta_j|$$

ridge (squares) shrinks every coefficient towards zero; lasso (absolute values) can push weak ones exactly to zero

- λ is the penalty for complexity — the bias–variance dial made explicit.
- **Ridge** tames collinear, noisy features by shrinking them together.
- **Lasso** also **selects**: it zeroes coefficients, leaving a simpler story.

When it earns its keep

On a handful of clean features, regularisation barely moves the needle. On hundreds of correlated ones, it's the difference between a model and a memoriser.

Regression in code

regress.py

python

```
from sklearn.linear_model import (  
    LinearRegression, Ridge, Lasso)  
for name, reg in [("linear", LinearRegression()),  
                  ("ridge", Ridge(alpha=1.0)),  
                  ("lasso", Lasso(alpha=0.1))]:  
    m = Pipeline([("prep", prep),  
                  ("reg", reg)]).fit(X_tr, y_tr)  
    score(name, m, X_te, y_te)  # R2, RMSE
```

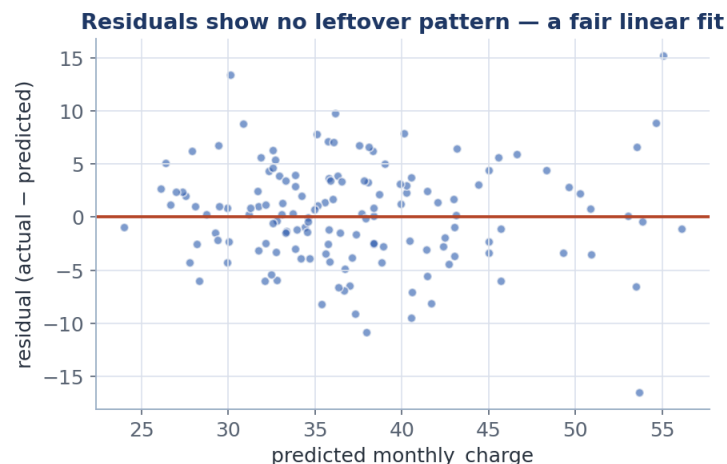
```
$ python regress.py
```

target = monthly_charge

	R2	RMSE
linear	0.668	4.74
ridge a=1.0	0.668	4.74
lasso a=0.1	0.670	4.72

Here the three barely differ — few features, little collinearity. That **sameness** is the finding, not a disappointment.

Reading a fit honestly



- $R^2 = 0.67$ says the model explains about two-thirds of the variation in charge.
- The residuals scatter evenly around zero with no curve or funnel — the linear form is fair here.

Don't stop at R^2

A high R^2 with a **patterned** residual plot is a warning, not a win. The picture catches what the number hides.

Section 3

Classification

Logistic regression (a classifier, despite the name)

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x)}}$$

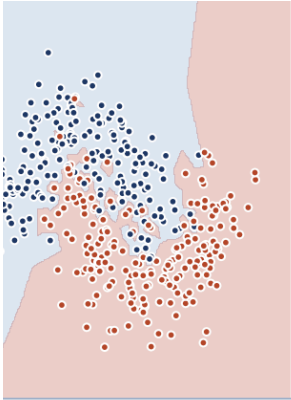
a linear score is squashed into a probability between 0 and 1; a threshold then turns the probability into a decision

- It models the **log-odds** of the positive class as a straight line.
- The output is a probability — richer than a bare label, and the raw material for every metric later.

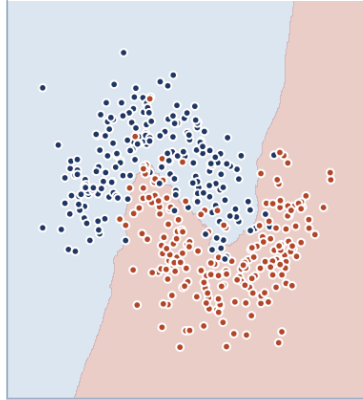
The boundary is linear, but the **threshold** on the probability is yours to set. Most of the interesting decisions live there, not in the fit.

k-NN: lazy, local, scale-sensitive

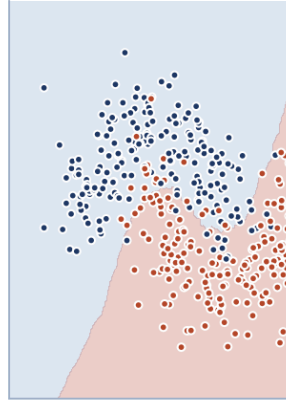
(jagged, high variance)



$k = 15$ (balanced)



$k = 60$ (smooth, higher bias)



- No training step: it just remembers the data and votes among the nearest neighbours.
- k is the capacity dial — $k = 1$ memorises every point; large k smooths everything into mush.

Week 1 pays off

Distances need comparable scales. Skip standardisation and income (in thousands) silently drowns out age.

How they think

- Ask one yes/no question at a time, each split making the groups purer (Gini or entropy).
- Read the path from root to leaf and you have the model's reasoning in plain sight.

Where they bite

- Grown unchecked, a tree draws a box around every training point — pure memorisation.
- Depth is the capacity dial; pruning or a depth cap is how you hold it back.

Trees are the most **interpretable** model here and among the easiest to overfit. Interpretability and reliability are not the same virtue.

Ensembles: why a crowd beats an expert

Bagging → random forests

- Grow many trees on resampled data and different feature subsets.
- Each is wrong in its own direction; averaging cancels the noise. Variance falls.

Boosting → gradient boosting

- Add small trees in sequence, each fixing the last one's mistakes.
- Bias falls, but greed can overfit — it needs a gentler hand.
- In practice: **XGBoost** / **LightGBM**, or scikit-learn's install-free **HistGradientBoosting** — the lab's choice.

The real lesson

Ensembles win by **diversity**, not brilliance. A hundred mediocre-but-different trees routinely beat one carefully tuned expert.

Support vector machines: the widest street

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^\top x_i + b))$$

find the boundary that leaves the widest gap between the classes; C trades margin width against tolerating the awkward points that sit inside it

- Don't just separate the classes — separate them with the **widest margin**. Only the points on the edge, the support vectors, actually shape the boundary.
- The **kernel trick** bends the space so a straight margin becomes a curved boundary — non-linear, without the polynomial bookkeeping.

Reach for it when

the margin is fairly clean, the sample is modest, or the features vastly outnumber the rows. Skip it on huge data, and remember it hands you distances, not calibrated probabilities. You'll benchmark it in the lab.

Classification in code

classify.py

python

```
models = {
    "logistic": LogisticRegression(max_iter=1000),
    "kNN":      KNeighborsClassifier(n_neighbors=15),
    "tree":     DecisionTreeClassifier(max_depth=4),
    "forest":   RandomForestClassifier(n_estimators=300),
}
for name, clf in models.items():
    m = Pipeline([("prep", prep),
                  ("clf", clf)]).fit(X_tr, y_tr)
    report(name, m, X_te, y_te)
```

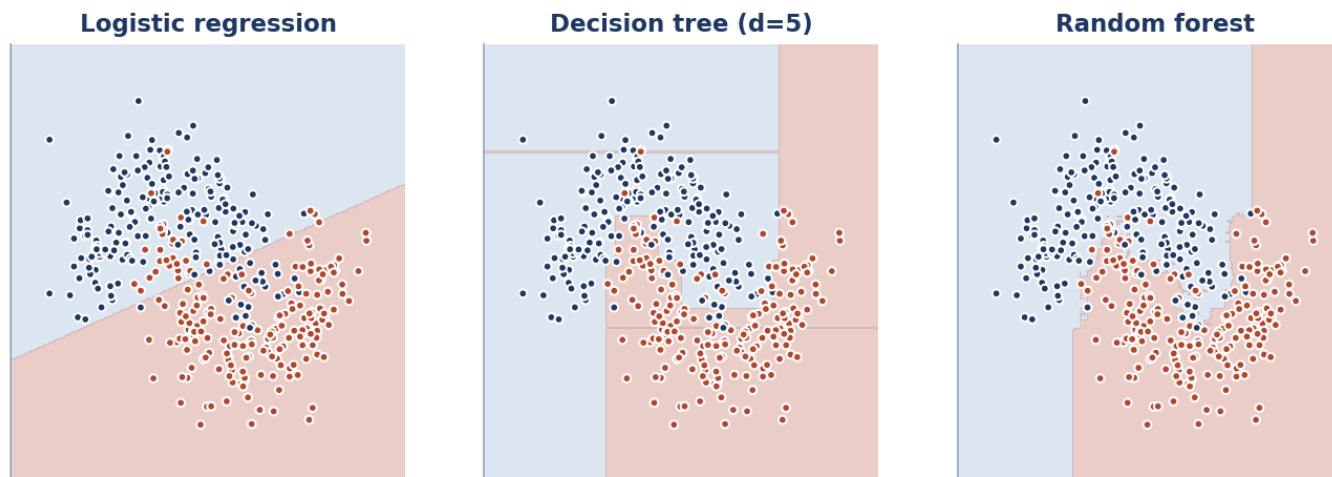
```
$ python classify.py
```

target = churn

	acc	prec	rec	F1	AUC
logistic	0.880	0.600	0.652	0.625	0.949
kNN k=15	0.873	0.643	0.391	0.486	0.933
tree d=4	0.900	0.643	0.783	0.706	0.931
forest	0.887	0.636	0.609	0.622	0.956

Notice kNN's recall: 0.39. It's as "accurate" as the rest and misses six in ten leavers. Hold that thought for the evaluation section.

Same data, three different minds



One dataset, three boundaries. Logistic insists on a straight cut; the tree carves rectangles; the forest blends many into a smooth, flexible frontier.

The right shape depends on the data's shape. “Which model is best?” is the wrong question; “best for what, measured how?” is the right one.

Section 4

Loss and the bias–variance trade-off

Loss: what you optimise is what you get

$$\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2 \quad \text{log-loss} = -\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

squared error for regression punishes big misses hardest; log-loss for classification punishes confident wrong answers

- Swap MSE for **MAE** and you stop letting a few outliers dominate the fit.
- The loss encodes your priorities before a single parameter is learned.

A design choice, not a default

“Which loss?” is really “which mistakes can I least afford?”. Answer that first; the optimisation just obeys.

Two ways to be wrong

Underfitting (too rigid)

- Misses the pattern even on the training data.
- Training and validation scores are both poor, and close together.
- The cure is **more** capacity.

Overfitting (too flexible)

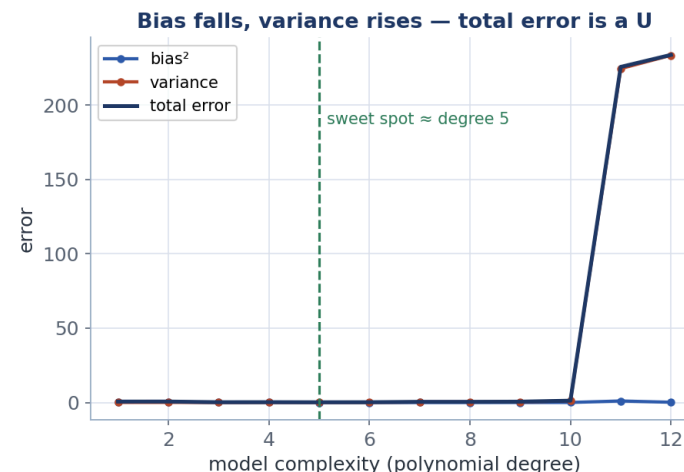
- Learns the noise; brilliant on train, poor on new data.
- A wide gap between training and validation scores.
- The cure is **less** capacity, or more data.

Same symptom — a bad validation score — two opposite cures. Guess wrong and you make it worse. The next two slides tell them apart.

The bias–variance trade-off

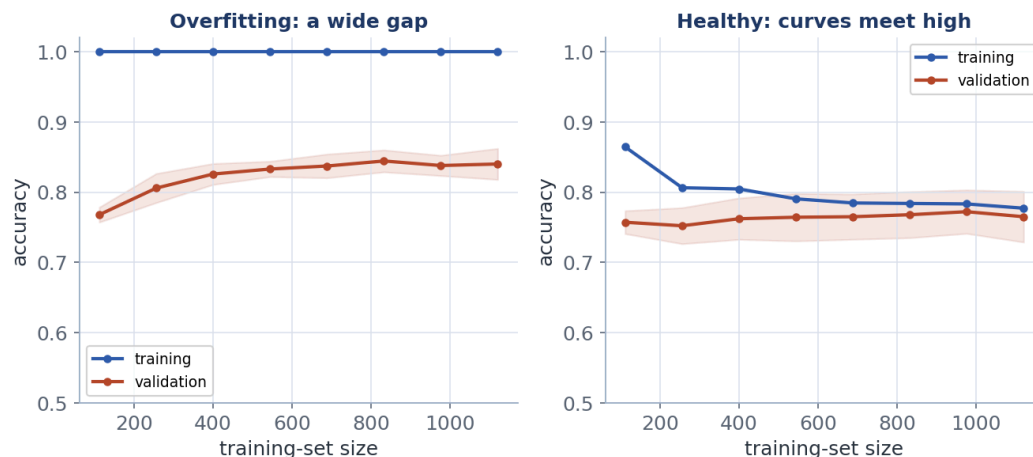
$$\mathbb{E}\left[(y - \hat{f})^2\right] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

- **Bias:** wrong assumptions, too simple.
- **Variance:** too twitchy, chases the sample.
- σ^2 : irreducible noise — no model beats it.



Total error is a U. The job is not zero bias or zero variance; it's finding the bottom of the curve.

Diagnosing with learning curves



Plot train and validation score against training-set size. A stubborn gap (left) is variance; two curves meeting at a low ceiling would be bias.

A gap that won't close → variance. Add data, simplify, or regularise.

Both curves flat and low → bias. Add features or capacity; more data won't help.

Section 5

Evaluation that doesn't flatter

One split lies; cross-validation doesn't

5-fold cross-validation: every row scores on a slice it never trained on

fold 1	val	train	train	train	train
fold 2	train	val	train	train	train
fold 3	train	train	val	train	train
fold 4	train	train	train	val	train
fold 5	train	train	train	train	val

- A single split hands you one number and a false sense of precision.
- k -fold rotates the held-out slice, so every point is judged once — you get a mean **and** a spread.

Cross-validate the pipeline

The scaler and imputer must be refit inside each fold. Fit them once on everything and you're back to Week 1's leakage.

Regression metrics: RMSE, MAE, R^2

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2} \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

RMSE is the typical error in the target's own units; R^2 is the share of variation the model explains, from 0 to 1

- **RMSE** speaks in euros or units — punishes large misses.
- **MAE** is the plain average error — kinder to outliers.
- **R^2** is unitless and comparable, but flatters on easy data.

When RMSE and MAE disagree sharply, a few big errors are doing the damage. That gap is a clue worth chasing.

Accuracy is the number that lies most

- Fraud runs at **3%** of transactions. A model that says “never fraud” scores **97%** accuracy and catches nothing.
- On churn, our models sit near 88% accuracy — but that hides a recall spread from 0.39 to 0.78.

the lazy model

```
prevalence of fraud = 3.4%  
"always say no fraud"  
    accuracy = 96.6%  
    frauds caught = 0
```

Rule of thumb

On any rare-event problem, quote accuracy and someone should stop you. It's the first metric to reach for and the first to mislead.

The confusion matrix

- Four outcomes, and the two **errors** rarely cost the same.
- A false negative on a tumour and a false positive on a spam filter are not remotely the same mistake.

Before any metric

Decide which error hurts more. The whole evaluation flows from that one judgement.

Confusion matrix — the two mistakes are not equal

true: stay	117	10
true: churn	8	15
	pred: stay	pred: churn

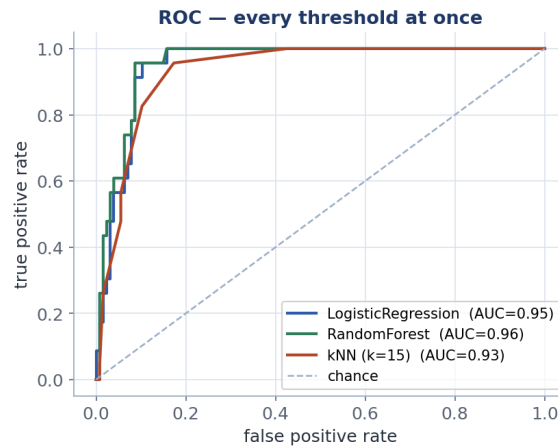
Precision, recall, and their tension

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

precision: of those we flagged, how many were real; recall: of the real ones, how many we caught

- Push recall up and precision usually falls — you flag more, and more of it is wrong.
- **F1** splits the difference, but “the difference” is only worth splitting if both errors cost the same.

Don't optimise F1 out of habit.
Optimise the error your problem can least afford, and let the other float.

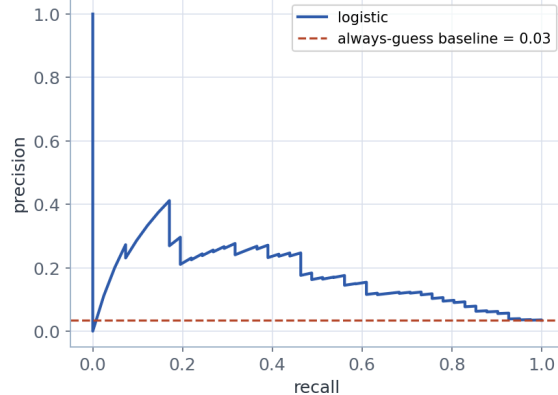


- The ROC sweeps **every** threshold at once; AUC is the area beneath it.
- AUC = the chance the model ranks a random positive above a random negative. 0.5 is a coin; our forest hits **0.96**.

AUC is threshold-free and great for **ranking**. It says nothing about where you should actually cut.

When ROC flatters: precision–recall

Precision–recall at 3% prevalence (accuracy of a lazy model = 97%)

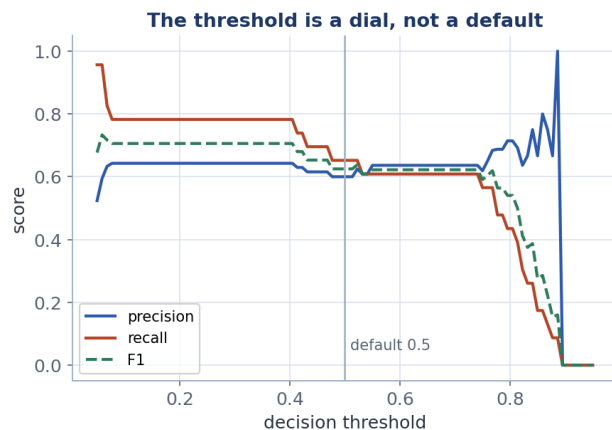


- At 3% prevalence, ROC curves look reassuringly high even when the model is nearly useless.
- The **precision–recall** curve stays honest, because it ignores the huge, easy negative class.

Pick the right curve

Rare positives → read PR-AUC, not ROC-AUC. Same model, very different verdict.

The threshold is a dial, not a default



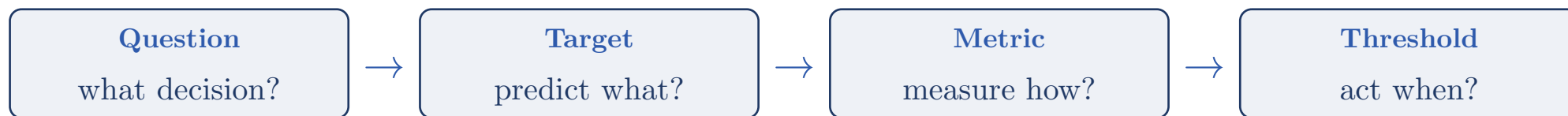
- 0.5 is a convention, not a law. Move it and precision and recall trade off continuously.
- Set it where the **costs** balance: aggressive for cancer screening, cautious for blocking a customer's card.

Choosing the threshold is a business decision wearing a maths costume. Own it explicitly.

Section 6

Framing real problems

The real skill: framing the question



- The algorithm is the easy, replaceable part. The chain above is where the judgement lives.
- Get the target and metric right and a plain logistic regression will beat a mis-aimed neural network every time.

A tour of applications

Problem	Frame as	The metric the question demands
Demand forecasting, pricing	regression	RMSE / MAE; asymmetric over- vs under-stocking costs
Credit scoring	classification	AUC and calibration; the cost of a default vs lost custom
Churn prediction	classification	recall on likely leavers, or the euro value of retention
Medical diagnosis	classification	recall first — a missed case is the dear mistake
Fraud and spam	classification	precision and PR-AUC; the inbox can't cry wolf
Predictive maintenance	classification	recall with enough lead time to act on it

Same handful of algorithms every time. What changes is the target and the metric — and those come from the domain, not the toolbox.

Two problems that pull opposite ways

Medical diagnosis

- A missed disease can be fatal; a false alarm means a follow-up test.
- So **recall** dominates. You accept more false positives to miss fewer cases.

Spam and fraud

- A blocked genuine email or card is a furious customer.
- So **precision** matters, and with rare positives you live on the PR curve.

Same maths, opposite dial

Nothing about the algorithm changed between these two. Only the answer to “which error can we least afford?” — and that flipped the whole evaluation.

From a prediction to a decision

- A probability is not an action. Someone still has to choose the threshold and what happens past it.
- Churn at 0.7 might mean “phone them”; at 0.4, “send a discount code”. The model informs; the policy decides.
- And data drifts — last year’s threshold quietly rots. That’s Week 1’s monitoring loop closing.

The model is one link in a chain that ends in a human decision and a real cost. Evaluate the chain, not just the classifier.

Choosing target and metric: a checklist

1. What **decision** does this feed, and what does a wrong one cost?
2. Is the honest target a **number** or a **category**?
3. Which **error** — false positive or false negative — hurts more here?
4. Is the positive class **rare**? If so, retire accuracy and ROC for PR.
5. Where should the **threshold** sit so the costs actually balance?

If you can answer these

you've done the hard 80% of the modelling before importing a single estimator.

1. **Train and compare a roster** — logistic, k-NN, decision tree, random forest, HistGradientBoosting and an SVM — all on top of your Week 1 pipeline.
2. **Diagnose** over- and underfitting with learning curves; act on what you find.
3. **Produce** a visual evaluation report, and argue which metric your chosen application demands — and why accuracy would mislead.

Deliverable

A runnable notebook plus a short written case for your metric choice, committed to your repo. Week 3 audits these very models for fairness.

- **The question sets the target, the target sets the metric, the metric sets the threshold.** Defaults decide none of these for you.
- **Every model has one real knob — capacity.** Bias–variance tells you which way to turn it; regularisation, depth and k are the knob.
- **Accuracy is the flattering number.** On rare events it's worthless — read precision, recall and the PR curve instead.
- **Cross-validate the whole pipeline.** One split is an anecdote; five folds are evidence.
- **Ensembles win by cancelling mistakes,** not by being individually clever.

Books

- James, Witten, Hastie, Tibshirani & Taylor — *An Introduction to Statistical Learning* (Python ed.). The clearest route into everything today.
- Hastie, Tibshirani & Friedman — *The Elements of Statistical Learning*. The deeper reference for regularisation and bias–variance.
- Géron — *Hands-On Machine Learning* (3rd ed.). Practical scikit-learn that mirrors the lab.

Papers and guides

- Breiman (2001), *Statistical Modeling: The Two Cultures* — read it to argue about what a model is **for**.
- Breiman (2001), *Random Forests* — the ensemble that still sets the bar.
- Saito & Rehmsmeier (2015), *The Precision–Recall Plot is More Informative than ROC on Imbalanced Data* — the evidence behind today’s imbalance argument.
- Provost, Fawcett & Kohavi (1998), *The Case Against Accuracy Estimation* — the case for distrusting accuracy.
- scikit-learn User Guide — *Model evaluation and Cross-validation*.